1. In the notebook, the audio waveform is transformed into time-frequency features before being passed to the keyword spotting model. What is the purpose of converting raw audio into spectrogram/MFCC-style features instead of directly using the raw time-domain waveform?

The reason why we convert raw audio into spectrogram/MFCC-style features is because the model recognizes the words easier. Because the raw audio wave form by itself does not show the frequencies that are present in the spoken word, it is not as useful to the model compared to the spectrogram/MFCC. This is because through spectrograms/MFCC, the features show how frequency energy changes over time. A spectrogram keeps both time and frequency information, and MFCCs compress that information into a unique signature of that specific word. This is useful for TinyML because the input becomes smaller and less noisy.

2. During full integer quantization, the notebook uses a representative dataset. What is the purpose of the representative dataset, and why is it important for creating an INT8 TensorFlow Lite model?

The purpose of the representative dataset is the fact that it gives the TensorFlow Lite model an example of what the actual audio features. This is important for creating an INT8 TensorFlow Lite model because these examples are used to estimate the minimum/maximum activation values in the model. If the representative dataset is poor or not similar to real test data, the model may classify the words “yes” or “no” incorrectly.

3. What does `audio_processor.get_data()` do? Refer to `input_data.py`.

audio_processor.get_data() extracts audio data from a particular dataset. Firstly, it loads an audio example, applies the preprocessing settings, and returns the processed feature data along with the correct labels.

4. In the final model export step, the TensorFlow Lite model is converted into a C/C++ byte array. What are `g_model`and `g_model_len`, and why do they need to be copied correctly into the Arduino source file?

g_model is the byte array that contains the actual TensorFlow Lite model data. g_model_len is the length of the array. Copying these values correctly is important because if the values are wrong, then TensorFlow lite will not be able to locate and load the model from the program memory. This may result in the model to not load, or just procure wrong predictions for the keyword.

5. Please include a screenshot showing the correct identification of at least one keyword, either **“YES”** or **“NO”**, as displayed in the Arduino IDE Serial Monitor after deploying your KWS model on the Arduino Nano 33 BLE. The screenshot should be included in your submitted PDF.